

SWIFT

AI-Native Exploit Chain Discovery & Governance Platform

Complete Project Guide, Execution Plan & Strategy

Timeline: 20 weeks (Part-time, 20-30 hrs/week)

Target: 3-5 customers, \$1-3K MRR

Budget: ~\$800-1000 (mostly API calls)

Status: Ready to Build

Created: April 17, 2026

Table of Contents

1. Part-Time Realistic Roadmap

Your actual plan with sustainable pace

2. Complete 12-Week Execution Plan

Foundation architecture and tech stack

3. Adaptive Access Model

GitHub OAuth, CLI, and On-Prem strategies

4. Competitive Analysis

How competitors solve code access problems

5. IP Protection Strategy

Protecting your moat while serving clients

6. Reality Check

Part-time vs full-time comparison

Quick Start (First Week)

TODAY: Set up Anthropic API account

<https://console.anthropic.com> (30 min)

TODAY: Create project folder & Python venv

`mkdir swift && python -m venv venv` (15 min)

TODAY: pip install anthropic

`pip install anthropic requests python-dotenv` (10 min)

Day 2-3: Write first Claude call (Haiku)

`scanners/claude_analyzer.py` (3 hrs)

Day 4-5: Test on public repo

Find small Python repo < 1000 LOC (2 hrs)

Total Week 1: ~5-6 hours actual work

SWIFT: Realistic Part-Time Roadmap

"20-30 hrs/week, Starting from Scratch, Building Smart"

Status: Adjusted for reality

Timeline: 16-20 weeks (not 12) — moving at sustainable pace

Deliverable: MVP that works, not perfect

HONEST ASSESSMENT

Your Constraints:

- Part-time (20-30 hrs/week) = ~4-6 hrs/day, 4-5 days/week
- Starting from scratch (no existing codebase)
- No API budget yet (need to request/set up)
- But you have 6 projects in 12 months (proven executor)

What This Means:

Full-time person:

Week 1: 40 hrs → significant progress

Part-time person:

Week 1: 25 hrs → real but slower progress

Real timeline:

12-week full-time plan → 16-20 week part-time plan

What You CAN'T Do:

- Ship polished product in 12 weeks
- Close 5 customers by Week 12
- Do daily customer support + building simultaneously
- Compete with full-time founders on speed

What You CAN Do:

- Build a working MVP in 16 weeks
- Validate the idea thoroughly
- Get 2-3 paying customers by Week 16-20
- Sustainable pace (no burnout)
- Keep your day job / other commitments

REFRAMED TIMELINE

Phase 1: Foundation (Weeks 1-4, ~100 hrs)

Goal: Get exploit chain discovery working on public repos

Week 1: Setup (25 hrs)

WHAT TO DO:

[] Set up Anthropic API account

- <https://console.anthropic.com>

- Create account
- Add payment method
- Get API key
- [] Set up GitHub account for testing
- Create new GitHub account (optional, cleaner)
- Fork 5 open-source repos (Python, JS, Go)
- Star them (bookmark for testing)
- [] Create local dev environment
- Create /swift project directory
- Create venv: `python -m venv venv`
- Create requirements.txt:


```
anthropic==0.28.0
requests==2.31.0
PyGithub==2.1.1
python-dotenv==1.0.0
```
- [] Create basic project structure
- /swift/scanners/claude_analyzer.py (empty)
- /swift/tests/test_analyzer.py (empty)
- /swift/.env (API_KEY=xxx)
- /swift/README.md (placeholder)
- TIME BREAKDOWN:
 - API setup: 30 min
 - GitHub setup: 15 min
 - Dev environment: 30 min
 - Project structure: 15 min
 - Total: ~1.5 hrs work
- BUFFERING FOR:
 - Troubleshooting: 2-3 hrs
 - Learning Anthropic docs: 2-3 hrs
 - Total actual time: ~6-8 hrs this week

Week 2: First Claude Call (25 hrs)

WHAT TO DO:

- [] Read Anthropic docs (1-2 hrs)
 - <https://docs.anthropic.com>
 - Understand token counting
 - Understand prompt engineering best practices
 - Understand rate limits
- [] Write scanners/claude_analyzer.py
 - Basic class structure

- First function: triage(code: str) -> List[str]
- Use Haiku for fast pattern matching
- Test on small code snippet (20 lines)

SKELETON:

```
import anthropic
import os

class ClaudeAnalyzer:
    def init(self):
        self.client = anthropic.Anthropic(apikey=os.getenv("ANTHROPICAPIKEY"))

    def triage(self, code: str, language: str) -> list:
        """Fast pattern matching with Haiku"""
        prompt = f"""Analyze this {language} code for security vulnerabilities.
        Look for: SQL injection, XSS, buffer overflow, eval(), pickle.loads()

        CODE:
        {code}

        Return ONLY a JSON list of suspicious lines:
        ["line 12: strcpy without bounds check", "line 45: SQL concatenation"]
        """

        message = self.client.messages.create(
            model="claude-3-5-haiku-20241022",
            maxtokens=500,
            messages=[{"role": "user", "content": prompt}]
        )
        return message.content[0].text
```

- [] Test on first repo
- Clone small Python repo (< 1000 lines)
- Run triage() on it
- See if Haiku finds issues
- Cost: ~\$0.01-0.05
- [] Document findings
- Screenshot output
- Note cost
- Note latency
- Save to /swift/tests/week2_results.txt

TIME BREAKDOWN:

- Reading docs: 2 hrs
- Writing code: 3 hrs
- Testing: 2 hrs
- Troubleshooting: 3-4 hrs
- Total: 10-11 hrs work

Week 3: Vulnerability Analysis (25 hrs)

WHAT TO DO:

- [] Extend to deep analysis
- Add function: analyze_vulnerability(location: str, code: str) -> dict
- Use Sonnet (more powerful)

- For each Haiku flag, do deep analysis

- Cost: ~\$0.50-1.00 per repo

SKELETON:

```
def analyzevulnerability(self, vulnlocation: str, codecontext: str) -> dict:
```

```
    """Deep analysis with Sonnet"""
```

```
    prompt = f"""You are a security expert.
```

```
    A vulnerability was flagged at: {vulnlocation}
```

```
    Code context:
```

```
    {codecontext}
```

```
    Analyze:
```

```
    1. What type of vulnerability?
```

```
    2. Can it be exploited?
```

```
    3. What's the impact?
```

```
    4. Severity (LOW/MEDIUM/HIGH/CRITICAL)?
```

```
    Return JSON: {{"type": "...", "exploitable": true/false, "impact": "...", "severity": "...}}
```

```
    """
```

```
    message = self.client.messages.create(
```

```
        model="claude-3-5-sonnet-20241022",
```

```
        maxtokens=1000,
```

```
        messages=[{"role": "user", "content": prompt}]
```

```
    )
```

```
    return json.loads(message.content[0].text)
```

```
    [ ] Test on 3 repos
```

- Run Haiku triage

- Run Sonnet analysis on findings

- Total cost: ~\$1-3 across 3 repos

- Document: vulnerabilities found, cost, latency

```
    [ ] Compare against Semgrep
```

- Install Semgrep: `pip install semgrep`

- Run: `semgrep --config=p/security-audit repo/`

- Compare findings

- Note: "SWIFT found X, Semgrep found Y, overlap Z"

TIME BREAKDOWN:

- Writing code: 3 hrs

- Testing: 4 hrs

- Semgrep comparison: 2 hrs

- Troubleshooting: 4-6 hrs

- Total: 13-15 hrs work

Week 4: Chain Discovery (25 hrs)

WHAT TO DO:

```
    [ ] Implement chain discovery
```

- Add function: `discover_chains(vulnerabilities: List[dict]) -> List[dict]`

- This is THE key differentiator

- Prompt Claude: "Given these vulns, what chains exist?"

SKELETON:


```
def discoverchains(self, vulnerabilities: list) -> list:
    """Find exploit chains"""
    vulnsummary = "\n".join([
        f"- {v['type']} at line {v['line']}: {v['description']}"
        for v in vulnerabilities
    ])

    prompt = f"""You are a security expert analyzing vulnerabilities.

    Found vulnerabilities:
    {vulnsummary}

    QUESTION: Can any of these vulnerabilities be CHAINED together?
    A chain is when combining 2+ vulns creates a more severe attack.
```

Example:

- Vuln A: SQL injection (allows DB query)
- Vuln B: Weak authentication (can bypass with SQL)
- Chain: SQL injection + weak auth = full database access

For the vulnerabilities above:

1. List all possible chains
2. For each chain, explain the attack path
3. Rate severity (individual vs chain)

Return JSON:

```
{
  "chains": [
    {
      "name": "Chain name",
      "vulnsinvolved": ["Vuln A", "Vuln B"],
      "attackpath": "Step 1 → Step 2 → Outcome",
      "individualseverity": "MEDIUM",
      "chainseverity": "CRITICAL",
      "explanation": "..."
    }
  ]
}
```

```
message = self.client.messages.create(
    model="claude-3-5-sonnet-20241022",
    maxtokens=2000,
    messages=[{"role": "user", "content": prompt}]
)
return json.loads(message.content[0].text)["chains"]
```

[] Test on 5 repos

- Run full pipeline: Haiku → Sonnet → Chains
- Cost per repo: ~\$2-4
- Total cost for 5: ~\$10-20
- Document findings

[] Create case study document

- Pick 1 repo with interesting chains
- Write it up: "SWIFT found X chains in Y repo"
- Include vulnerability list + chains + attack paths
- This becomes your Week 5-6 talking point

TIME BREAKDOWN:

- Writing code: 4 hrs
- Testing: 5 hrs

- Creating case study: 3 hrs
- Troubleshooting: 5-8 hrs
- Total: 17-20 hrs work

Phase 2: Validation (Weeks 5-8, ~100 hrs)

Goal: Prove chains work on real code, get customer feedback

Week 5-6: Customer Conversations (40 hrs)

WHAT TO DO:

[] Prepare

- Write email pitch (2 min read)
- Link to case study
- Link to demo

PITCH TEMPLATE:

Subject: We found exploit chains in [Popular Project] - you might care

Hi [Name],

We built SWIFT, a tool that finds exploit chains (not just bugs).

Most tools find individual vulnerabilities. We find attack paths.

Example: We scanned [Popular Project] and found a 3-step chain:

1. SQL injection (line 42)
2. Authentication bypass (line 156)
3. Privilege escalation (line 203)

Individually, these are medium severity.

Chained: Critical RCE.

Most tools catch 1-2. We catch the chain.

If interesting, we're looking for early feedback.

Demo: [link to case study]

Curious?

-[Your Name]

[] Find 10 prospects

- Twitter/LinkedIn security folks
- CTF community (HackTheBox, TryHackMe)
- Startup founders (security-conscious)
- Product Hunt security products

[] Send 10 cold emails

- Expect 20-30% response rate (2-3 replies)
- Schedule calls with responders

[] Conduct 3-5 customer calls (30 min each)

- Ask: "Is this valuable?"
- Ask: "Would you pay for this?"

- Ask: "What's missing?"
 - Take notes
- TIME BREAKDOWN:
- Pitch writing: 1 hr
 - Research prospects: 2 hrs
 - Email outreach: 2 hrs
 - Call prep: 2 hrs
 - Calls (3x30min): 1.5 hrs
 - Note-taking + analysis: 2 hrs
 - Total: ~10-12 hrs for Week 5
 - Repeat in Week 6: ~10-12 hrs
 - Total Phase 2: ~20-24 hrs

Week 7-8: Iterate Based on Feedback (60 hrs)

- WHAT TO DO:
- ☐ Analyze feedback
 - What did customers care about?
 - What did they NOT care about?
 - Adjust product roadmap
 - ☐ Add first non-core feature
 - From customer feedback
 - 10-15 hrs of work
 - E.g., "better formatting", "export PDF", "Slack integration"
 - ☐ Polish presentation
 - Create landing page (Webflow or simple HTML)
 - Write 1-2 blog posts
 - Prepare HN post draft
 - ☐ Set up infrastructure for paying customers
 - Stripe account (for payments)
 - Email template for onboarding
 - Basic invoicing
- TIME BREAKDOWN:
- Analysis + iteration: 5 hrs
 - Feature implementation: 15 hrs
 - Landing page: 10 hrs
 - Blog post: 5 hrs
 - Stripe setup: 3 hrs
 - Total: ~38-40 hrs for Weeks 7-8

Phase 3: Public Launch (Weeks 9-12, ~80 hrs)

Goal: Get community attention, get first customers

Week 9: GitHub OAuth Integration (25 hrs)

WHAT TO DO:

This is complex, so SIMPLIFY for part-time:

OPTION A (Easier, 10 hrs):

- Create GitHub API script
- Manual GitHub OAuth (for testing)
- Clone repo via GitHub API
- Scan it
- Delete it
- Don't build full UI yet

OPTION B (Full, 25 hrs):

- Build OAuth flow in FastAPI
- Create dashboard UI
- Full integration

RECOMMENDATION: Do OPTION A for Week 9

- Proves concept
- Less code
- Customer can use CLI instead

SKELETON (Option A):

```
from github import Github

class GitHubIntegration:
    def scanrepo(self, repourl: str, apikey: str):
        """Clone from GitHub, scan, delete"""
        # repourl: "owner/repo"
        g = Github(apikey)
        repo = g.getuser(repourl.split('/')[0]).getrepo(repourl.split('/')[1])

        # Clone (simulate)
        # Scan
        # Return results
```

TIME: 10-15 hrs for part-time (keep it simple)

Week 10: CLI Polish + Testing (25 hrs)

WHAT TO DO:

- [] Finalize CLI tool
- swift scan --repo . --api-key xyz
- Make it user-friendly
- Add error handling
- Create README with examples
- [] Create installation guide
- pip install swift-cli
- or: python setup.py install

[] Test on 10 public repos

- Different languages
- Different sizes
- Document findings

TIME: 15-20 hrs

Week 11: Public Launch (20 hrs)

WHAT TO DO:

[] Write HN post

- "We found exploit chains that Veracode misses"
- Include 2-3 real examples from public repos
- 500-800 words
- Post Monday morning (best day)

[] Engage 24 hours

- Answer comments
- Ask for feedback
- Stay online

[] Monitor metrics

- HN points (goal: 50+)
- Email signups (goal: 20+)
- GitHub stars if public (goal: 50+)

TIME: 8-10 hrs (but spread across the week)

Week 12: First Customers (15 hrs)

WHAT TO DO:

[] Reach out to Week 5-6 prospects

- "We're live! Want to try?"
- Offer: 2-week free trial

[] Create simple pricing

- Free: 5 scans/month (demo)
- Paid: \$299/mo (unlimited)
- Or: Pay-per-scan \$50/scan

[] Get first 1-2 customers

- Manual onboarding
- Hand-hold through first scan
- Get feedback

TIME: 10-15 hrs (customer calls + setup)

REVISED TIMELINE

Phase 1: Foundation (Weeks 1-4)

Week 1: Setup API + environment

Week 2: First Claude call (Haiku triage)

Week 3: Deep analysis (Sonnet)

Week 4: Chain discovery (THE KEY FEATURE)

MILESTONE: Exploit chains working on public repos

SUCCESS METRIC: Find 1+ real chain per repo

Phase 2: Validation (Weeks 5-8)

Week 5-6: Talk to 5 customers

Week 7-8: Iterate + polish

MILESTONE: Customer feedback + adjusted product

SUCCESS METRIC: 3+ people say "this is valuable"

Phase 3: Launch (Weeks 9-12)

Week 9: GitHub OAuth (simple version)

Week 10: CLI polish + testing

Week 11: Public launch (HN)

Week 12: First customers

MILESTONE: Public proof + first revenue

SUCCESS METRIC: \$0-1K MRR + 50+ email signups

Phase 4: Scaling (Weeks 13-20)

Week 13-16: Add features based on customer feedback

Week 17-20: Double down on sales + customer success

MILESTONE: 2-5 paying customers

SUCCESS METRIC: \$2-5K MRR

WHAT TO SKIP (Part-Time Reality)

Skip these for now:

- ■ Full GitHub OAuth UI (too complex for part-time)
- Instead: Simple API calls, manual GitHub auth
- ■ On-prem deployment (too early, enterprise feature)
- Instead: Focus on SaaS + CLI
- ■ Advanced governance layer (nice-to-have, not MVP)
- Instead: Simple permission model + basic audit logs
- ■ Docker sandbox (complex, not required for MVP)
- Instead: Validate patches manually in Week 15+
- ■ Perfect code quality (ship, iterate, improve)
- Instead: Get it working first, optimize later

Keep these (MVP essentials):

- ■ Haiku triage (fast pattern matching)

- ■ Sonnet analysis (deep reasoning)
- ■ Chain discovery (THE differentiator)
- ■ CLI tool (easy for customers)
- ■ Public case studies (proof of concept)
- ■ Landing page (credibility)
- ■ Stripe integration (payments)

REALISTIC DELIVERABLES BY TIMELINE

By Week 4:

- Working Claude analyzer (Haiku + Sonnet)
- Chain discovery on 5 public repos
- Case study document
- **Cost**: ~\$20-30 in API calls

By Week 8:

- Customer feedback + validated interest
- CLI tool (basic, working)
- Landing page
- **Cost**: ~\$50-100 in API calls (testing)

By Week 12:

- Public launch (HN post)
- 1-2 paying customers
- \$299-999/mo revenue (1-2 customers)
- 30-50 email signups
- **Cost**: ~\$200 in API calls (production scanning)

By Week 20:

- 3-5 paying customers
- \$2-5K MRR
- GitHub OAuth integration
- CLI tool polish
- **Cost**: ~\$500-1000 in API calls (production + development)

REALISTIC BUDGET

Phase 1 (Weeks 1-4):

API calls: \$30

Anthropic credits: \$5 (already free)

Total: \$35

Phase 2 (Weeks 5-8):

API calls: \$100 (customer testing)

Domain name: \$12/year

Total: \$112

Phase 3 (Weeks 9-12):

API calls: \$200 (production scanning)

Stripe: \$0 (2.9% on transactions)

```
Total: $200

Phase 4 (Weeks 13-20):

API calls: $500 (customer base)
Hosting: $0 (could be free initially)

Total: $500

GRAND TOTAL: ~$800-1000 for 20 weeks
```

Plus: Anthropic \$5 free credits (covers Weeks 1-2 entirely)

WEEKLY TIME ALLOCATION (20-30 hrs/week)

Ideal week breakdown:

```
Monday-Wednesday (12-18 hrs):

- Core development work (4-6 hrs/day)
- Building features
- Testing
- No distractions

Thursday (3-5 hrs):

- Customer communication
- Feedback analysis
- Administrative

Friday (2-3 hrs):

- Planning next week
- Documentation
- Reflection
```

Sample Week 3 Schedule:

```
Monday: 5 hrs (write analyze_vulnerability function)
Tuesday: 5 hrs (test on 3 repos, fix bugs)
Wednesday: 4 hrs (integrate Semgrep comparison)
Thursday: 3 hrs (document findings, prepare for Week 4)
Friday: 2 hrs (plan Week 4 chain discovery)
Total: 19 hrs (realistic for part-time)
```

HONEST EXPECTATIONS

What's realistic:

- ■ Working MVP by Week 16

- ■ 2-3 paying customers by Week 20
- ■ \$1-3K MRR by end of timeline
- ■ Sustainable pace (no burnout)
- ■ Clear validation of idea

What's NOT realistic (part-time):

- ■ 5 customers by Week 12
- ■ \$5K+ MRR by Week 12
- ■ Polished, production-ready product
- ■ Competing with full-time founders on speed
- ■ Daily customer support from Day 1

NEXT STEPS (This Week)

TODAY:

```
[ ] Set up Anthropic account
- Go to https://console.anthropic.com
- Create account
- Add payment method ($10-20 for testing)
- Copy API key to .env file
[ ] Create project folder
- mkdir swift
- cd swift
- python -m venv venv
- source venv/bin/activate (or venv\Scripts\activate on Windows)
[ ] Create requirements.txt
anthropic==0.28.0
requests==2.31.0
python-dotenv==1.0.0
[ ] pip install -r requirements.txt
[ ] Create first file: scanners/claude_analyzer.py (empty for now)
TIME: 1.5-2 hrs max
```

By End of Week 1:

```
[ ] Read Anthropic docs (skim, 30 min)
[ ] Write first Claude call (Haiku triage)
[ ] Test on 1 small code sample
[ ] Cost: ~$0.01-0.05
If this works → confidence boost → start Week 2
If this fails → debug → typical, keep going
```

REALITY CHECK

You're building:

- Not a perfect product ✗
- A working proof of concept ✓

You're validating:

- Not "can we scale to 1M ARR?" (too early)
- "Do customers value exploit chains?" ✓

Your goal:

- Not "beat Veracode in 12 weeks" (impossible)
- "Get 2-3 customers, \$1-3K MRR, clear validation" ✓

FINAL ADVICE

Part-time from scratch is HARD but DOABLE.

The key: Focus ruthlessly.

Each week, ask: "Does this move me closer to customer feedback?"

- Yes → Do it
- No → Skip it

Examples:

- "Should I build perfect GitHub OAuth?" → NO (use manual OAuth for now)
- "Should I create case study on public repo?" → YES (credibility)
- "Should I setup Docker sandbox?" → NO (customers don't care yet)
- "Should I talk to 5 customers?" → YES (validation)

Ship the minimum. Talk to customers. Iterate.

That's the part-time playbook.

Document created: April 17, 2026

Status: Ready for part-time execution

Next checkpoint: April 28, 2026 (end of Week 1) — API working?

SWIFT: 12-Week Execution Plan

Status: Ready to Ship
Start Date: April 21, 2026
MVP Deadline: July 10, 2026

1. CORE DECISIONS (LOCKED IN)

AI Model Strategy

- **Development**: Claude 3.5 Opus (building the platform)
- **Production Scanning**:
 - **Haiku** → Fast triage (pattern matching, \$0.05-0.10/repo)
 - **Sonnet** → Deep analysis (chain reasoning, \$0.50-2.00/repo)
- **Decision logic**: Client determines tier based on budget + risk tolerance
- Example: Startup = Haiku-only (\$0.10/scan), Mid-market = Haiku→Sonnet hybrid (\$2-3/scan)

Patch Workflow

```
Claude generates 3-5 patch candidates
↓
Sandbox validates (Docker isolation)
↓
System flags safest + most maintainable
↓
[HUMAN APPROVAL REQUIRED] ← Client team reviews
↓
Patch applied or rejected
↓
Audit log records decision + approver
```

Market Entry

- **Primary**: Startups (Series A-C, \$2M-\$20M ARR)
- Fast sales cycle (2 weeks)
- GitHub-native teams
- Price point: \$299-999/mo
- **Secondary** (month 4+): Mid-market after proof points
- **Tertiary** (month 8+): Enterprise (compliance-heavy)

Go-To-Market

- **Closed source** initially (maintain competitive edge on chain discovery)
- **Future**: Consider open-source components (e.g., governance layer) for developer mindshare

2. TECH STACK (FINALIZED)

Backend

Language: Python 3.11+

Framework: FastAPI (REST API) + async support

Package Mgr: Poetry

Testing: pytest + pytest-asyncio

Core Components

Infrastructure

Development: Local Docker + Python venv

Testing: GitHub Actions (CI/CD)

Staging: Docker Compose (local simulation)

Production: Docker + Docker Registry (customer-hosted or SaaS)

Database/Storage

- **Audit Trail**: SQLite (local) → PostgreSQL (production)
- **Vulnerability Cache**: JSON files + optional Redis
- **Patch History**: Git-based versioning

Folder Structure

```
/swift/  
  
└── agent/  
    ├── controller.py [Main orchestrator]  
    ├── state_machine.py [Workflow states]  
    └── scanners/  
        ├── claude_analyzer.py [MISSING - Week 1 task]  
        ├── semgrep_runner.py [Existing]  
        └── chain_discoverer.py [New - Week 2]  
    └── remediation/  
        ├── patch_generator.py [Claude API calls]  
        ├── sandbox_validator.py [Docker orchestration]  
        └── approval_workflow.py [Human-in-loop]  
    └── governance/  
        ├── permission_engine.py [RBAC + default-deny]  
        ├── forensic_logger.py [HMAC audit trail]  
        └── behavior_monitor.py [Claude anomaly detection]  
    └── api/  
        ├── routes.py [FastAPI endpoints]  
        ├── auth.py [API key + request signing]  
        └── models.py [Pydantic schemas]
```

```
■■■ cli/

■ ■■■ main.py [Command-line interface]

■■■ config/

■ ■■■ defaults.py [Constants]

■ ■■■ policies.json [Governance rules]

■■■ test/

■ ■■■ test_analyzer.py

■ ■■■ test_chains.py

■ ■■■ test_sandbox.py

■■■ requirements.txt [Python dependencies]

■■■ docker-compose.yml [Local dev environment]

■■■ README.md [Project docs]
```

3. ARCHITECTURE DIAGRAM



[illegible]

- [] Create `scanners/chain\_discoverer.py` (orchestration)
- [] Test on 5 repos, document chains found
- [] Create test cases (manual + automated)

Success Metric: Claude discovers 1+ exploit chain per repo that Semgrep misses

Deliverable: Working Python module + test results + 5 repo analysis

Go/No-Go Decision: If chains found → GO to Week 3 | If no chains → DEBUG and extend

****WEEKS 3-4: PATCH GENERATION + SANDBOX + END-TO-END DEMO****

Goal: Full pipeline working (vuln → patch → test → output)

Week 3 Tasks

```
• [ ] Create `remediation/patch_generator.py`:
```python
def generatepatch(vulncode: str, vulnerabilitydesc: str) -> List[str]
Return 3-5 patch candidates as unified diffs

def rankpatches(patches: List[str], testresults: dict) -> str
Return safest + most maintainable
```
```

```
• [ ] Create `remediation/sandbox_validator.py`:
```python
def testpatch(code: str, patch: str, tests: List[str]) -> dict
Docker: apply patch, run tests, check for regressions
Return {status, testresults, sideeffects}
```
```

- [] Wire into controller.py
- [] Test patch generation on 3 repos

Week 4 Tasks

- [] Integrate approval workflow (`remediation/approval\_workflow.py`)
- Generate approval request JSON
- Mock human reviewer (for demo)
- Log decision to audit trail
- [] Create evidence bundle generator
- [] Build CLI: `swift scan`
- [] ****End-to-end demo****: Upload repo → get vulns + chains + patches + audit log in 5 min

Success Metric: Demo works end-to-end, 0 crashes

Deliverable: Demo video + sanitized audit trail example

Go/No-Go Decision: If demo works → GO to customer conversations | If broken → DEBUG first

****WEEKS 5-6: CUSTOMER VALIDATION****

Goal: Talk to 5 startup security people, validate governance + pricing

Week 5 Tasks

- [] Identify 5 target startups (Series A-C, GitHub-heavy, security-conscious)
- Product Hunt, AngelList, HackerNews, Twitter security folks
- CTF communities (HackTheBox, TryHackMe)
- [] Reach out: "Building security scanning tool, want feedback?" + demo link
- [] Schedule 4-5 calls (30 min each)

Week 6 Tasks

- [] Conduct interviews:
 - "Does exploit chain discovery matter to you?"
 - "Would you pay \$299/mo for this?"
 - "Is audit trail important for your compliance?"
 - "What's missing?"
- [] Document feedback + iterate product
- [] Identify 2-3 best prospects for Week 11 outreach

Success Metric: 3+ people say "This is valuable" + 1+ wants trial

Deliverable: Feedback summary + product iteration backlog

Go/No-Go Decision: If strong interest → GO to launch prep | If meh → REFRAME value prop

****WEEKS 7-8: POLISH + LAUNCH PREP****

Goal: Production-ready code + marketing materials

Week 7 Tasks

- [] Code cleanup:
- [] Type hints everywhere
- [] Comprehensive docstrings
- [] Error handling + retry logic
- [] Logging at every step
- [] Create documentation:
- [] Architecture overview
- [] API docs (FastAPI auto-generates)
- [] Installation guide
- [] Quick start (5-min demo)
- [] Security audit (self + ask peer)

Week 8 Tasks

- [] Build landing page (single-page, no analytics yet)
- Problem → Solution → Why SWIFT → Pricing → CTA
- [] Write technical blog post:
- "How we find exploit chains that Veracode misses"
- Explain reasoning layer
- Include real example from customer repos
- [] Prepare HN/Product Hunt posts (pre-write, schedule)
- [] Set up email capture (landing page + Twitter link)

Success Metric: Code passes peer review, landing page live

Deliverable: Docs + blog post draft + landing page

****WEEKS 9-10: PUBLIC LAUNCH + COMMUNITY****

Goal: Get to 100+ HN points, 50 email signups

Week 9 Tasks

- [] ****Monday****: Post on Hacker News
- Title: "SWIFT: Finding Exploit Chains (not just bugs) in your code"
- Include real example + technical depth
- Engage first 24 hours (answer all questions)
- [] ****Tuesday****: Post on Product Hunt
- Nice visuals + demo video
- Daily engagement
- [] ****Wednesday-Friday****: Twitter threads explaining chains + governance

Week 10 Tasks

- [] Monitor feedback, iterate quickly
- [] Tweet wins: "100+ stars on HN! Thank you to..."
- [] Engage with security community (follow-ups, discussions)
- [] Collect early user feedback (from email signups)
- [] Plan Week 11 outreach (warm introductions from community)

Success Metric: 100+ HN points, 50+ email signups, 500+ Twitter impressions

Deliverable: Launch post + engagement metrics

****WEEKS 11-12: FIRST CUSTOMERS****

Goal: Close 2-5 paying customers, hit \$2-5K MRR

Week 11 Tasks

- [] Targeted outreach (50 prospects):
- Warm intros from Week 5 conversations
- CTF community leaders
- Startup founders (Twitter, LinkedIn)
- Security engineers (Angel List, LinkedIn)
- [] Pitch: "We found X vulns in your repos, including chains. Want help fixing them?"
- [] Offer: 2-week free trial + dedicated support
- [] Aim for 10+ calls scheduled

Week 12 Tasks

- [] Sales calls (30-60 min each)
- Demo SWIFT on their actual repo (if public)
- Explain ROI: "Find chains in 1 hour vs \$500K pentest"
- Close on 2-5 customers
- [] Onboard first customers:
- Set up repos
- Run first scan
- Review findings together
- Get feedback for product iteration
- [] Sign contracts (simple 1-pager okay for MVP)

Success Metric: 2-5 customers, \$2-5K MRR, positive feedback

Deliverable: Customer contracts + onboarding notes

5. PRICING STRATEGY

Startup (Series A-C)

- **Tier 1 (Free Trial)**: 5 repos, 1 month, Haiku-only
- **Tier 2 (Starter)**: \$299/mo, 20 repos/mo, Haiku triage
- **Tier 3 (Pro)**: \$999/mo, unlimited repos, Haiku→Sonnet hybrid
- **Tier 4 (Enterprise)**: \$5K+/mo, custom SLAs, dedicated support

Mid-Market (starting Week 13)

- **Team**: \$3K/mo, 100 repos/mo, Sonnet + on-prem option
- **Enterprise**: \$10K/mo, unlimited, governance + custom policies

6. KEY RISKS & MITIGATION

7. SUCCESS CRITERIA (GO/NO-GO GATES)

Week 2 Gate

- **■** Claude discovers real exploit chains in test repos
- **■** Chain complexity > simple bug chaining
- **Decision**: GO → Week 3 | PIVOT → redesign prompt strategy

Week 4 Gate

- ■ End-to-end demo works (no crashes)
- ■ Audit trail is immutable
- ■ Patch generation succeeds on 70%+ of vulns
- **Decision**: GO → customer validation | DEBUG → fix issues

Week 6 Gate

- ■ 3+ customers interested
- ■ Governance/audit trail resonates
- ■ Pricing feedback validated
- **Decision**: GO → launch | REFRAME → adjust positioning

Week 10 Gate

- ■ 100+ HN points (or equivalent engagement)
- ■ 50+ email signups
- ■ Community momentum
- **Decision**: GO → Week 11 outreach | REASSESS → improve messaging

Week 12 Gate

- ■ 2-5 paying customers
- ■ \$2-5K MRR achieved
- ■ Positive onboarding experience
- **Decision**: GO → scale | PIVOT → B2B vs product model

8. DAILY STANDUP TEMPLATE

WHAT I DID:

- [Task 1] Done/In progress
- [Task 2] Blocked by [X]

WHAT I'M DOING TODAY:

- [Priority task]
- [Secondary]

BLOCKERS:

- None / [Specific issue]

WINS:

- [Good thing happened]

9. RESOURCES NEEDED

API Costs (12 Weeks)

- **Anthropic API**: \$100 free credits + ~\$500-1000 additional
- **GitHub API**: Free (standard tier)
- **Docker**: Free (community)
- **Total**: ~\$1000-1500

Time Commitment

- Weeks 1-4: 60 hrs/week (core build)
- Weeks 5-6: 50 hrs/week (customer calls)

- Weeks 7-8: 45 hrs/week (polish)
- Weeks 9-10: 40 hrs/week (launch + community)
- Weeks 11-12: 50 hrs/week (sales)
- ****Total****: ~540 hours = ~2.6 full-time developer-months

Tools

- Python IDE (VSCode)
- Docker Desktop
- Git + GitHub
- Anthropic API account
- Landing page builder (Webflow or simple HTML)

10. NEXT IMMEDIATE ACTIONS

TODAY (End of call)

- [] Get Anthropic API key + set billing
- [] Create `/swift/scanners/claude_analyzer.py` skeleton`
- [] Identify 3 test repos (Python, JavaScript, Go)

This Week (April 21-25)

- [] Implement ``triage()` function (Haiku)`
- [] Implement ``analyze_vulnerability()` (Sonnet)`
- [] Test on first repo
- [] Document findings

By End of Week 2

- [] Exploit chain discovery working
- [] 5 repo analysis complete
- [] Go/No-go decision made

11. FINAL THOUGHTS

You have:

- ■ Strong security background
- ■ Proven execution (6 projects in 12 months)
- ■ Architecture already built
- ■ Governance moat (competitors can't copy in 18 months)
- ■ Market validation (5 customers by Week 6)
- ■ Realistic timeline (not a 24-month slog)

You're missing:

- ■ Claude API integration (2 weeks of work)

The plan above removes all ambiguity. It's a straight path from Week 1 → Week 12 → paying customers.

The only variable is execution velocity.

If you ship Week 1 deliverables on schedule → confidence skyrockets.

If you slip → adjust, but keep shipping.

Ship by Friday. Decide by Week 2.

Document Created: April 17, 2026

Status: Ready for Development

Next Checkpoint: Week 2 (April 28-May 2, 2026) - Chain Discovery Validation

SWIFT: Adaptive Access Model

"Meet Clients Where They Are"

...

The Model

CLIENT ARRIVES

SWIFT asks: "How would you like to scan?"

123456789101112131415161718192021222324252627282930313233343536373839404142434445464748495051525354555657585960616263646566676869707172737475767778798081828384858687888990919293949596979899100

■ Option 1: GitHub OAuth (Easiest) ■

■ - For: Startups, open-minded teams ■

■ - Cost: \$299-999/mo ■

■ - Trust: Medium (standard SaaS) ■

- - Setup: 2 minutes ■

Journal Pre-proof

■ Option 2: CLI Tool (Local-First) ■

- - For: Security-conscious devs ■

■ - Cost: \$299-999/mo (same) ■

■ - Trust: High (code never uploaded) ■

■ - Setup: 5 minutes ■

[illegible]

■ Option 3: On-Premise (Maximum Trust) ■

- - For: Enterprise, compliance-heavy ■

- - Cost: \$5K-50K/mo ■

- - Trust: Maximum (code stays in network)■

■ - Setup: 1-2 days ■

123456789101112131415161718192021222324252627282930313233343536373839404142434445464748495051525354555657585960616263646566676869707172737475767778798081828384858687888990919293949596979899100

Client chooses based on:

- Risk tolerance
- Compliance requirements
- Team size
- Budget

SWIFT adapts workflow

— — —

ACCESS MODEL DETAILS

****OPTION 1: GitHub OAuth** ■ (Default)**

Who Uses This:

- Series A-C startups (most)
- GitHub-native teams
- Fast decision-makers
- Budget: <\$1M ARR
- Risk tolerance: Medium

How It Works:

Flow:

1. User lands on SWIFT dashboard
2. Clicks "Connect GitHub"
3. GitHub OAuth prompt: "Allow SWIFT to read repos?"
4. User approves
5. SWIFT gets temporary access token
6. For each scan:
 - Clone repo from GitHub to temp directory
 - Run analysis (chain discovery + patches)
 - Delete temp directory
 - Send results to dashboard
 - Log in audit trail

Data Journey:

GitHub (your code)

↓ [SWIFT clones via OAuth]

SWIFT temp server (analysis happens here)

↓ [Code deleted immediately after]

SWIFT dashboard (results only)

↓ [User views findings]

Implementation (Week 3):

```
# scanners/github_integration.py

import os
import tempfile
import subprocess

from github import Github
from pathlib import Path

class GithubIntegration:
    def __init__(self, user_token: str):
        """Initialize with user's GitHub OAuth token"""

        self.gh = Github(user_token)
        self.user_token = user_token
```

```

def get_repo_and_analyze(self, repo_full_name: str):
    """
    Clone repo from GitHub, analyze, delete.
    repo_full_name: "owner/repo"
    """
    # Create temp directory
    with tempfile.TemporaryDirectory() as temp_dir:
        try:
            # Clone via OAuth
            repo_url = f"https://{self.user_token}@github.com/{repo_full_name}.git"
            subprocess.run([
                "git", "clone", "--depth", "1", repo_url, temp_dir
            ], check=True, capture_output=True)
            # Log clone action
            self.audit_log(f"Cloned {repo_full_name} via OAuth")
            # Analyze code
            code_content = self._read_repo_files(temp_dir)
            analyzer = ClaudeAnalyzer()
            vulnerabilities = analyzer.analyze_code(code_content, language="multi")
            chains = analyzer.discover_chains(vulnerabilities)
            patches = analyzer.generate_patches(vulnerabilities)
            # Log analysis
            self.audit_log(f"Analyzed {repo_full_name}: {len(chains)} chains found")
            # Temp directory auto-deleted by context manager
            # Code is GONE after this block
            return {
                "repo": repo_full_name,
                "vulnerabilities": vulnerabilities,
                "chains": chains,
                "patches": patches,
                "method": "github_oauth"
            }
        except Exception as e:
            self.audit_log(f"Error analyzing {repo_full_name}: {str(e)}")
            raise

    def list_user_repos(self):
        """List repos user has access to"""
        user = self.gh.get_user()
        return [(repo.name, repo.full_name) for repo in user.get_repos()]

```

```
def audit_log(self, action: str):  
    """Log to forensic audit trail"""  
  
    # Implemented in governance layer  
  
    pass
```

Client Pitch:

> "Connect your GitHub account. We'll scan your repos using GitHub's OAuth. Your code is analyzed on our servers and immediately deleted. Results show in your dashboard. Standard SaaS security—same as Snyk or GitHub Advanced Security."

Concerns Addressed:

- "Is our code safe?" → Code deleted after each scan
- "Can you see our code?" → Only during scan, then deleted
- "Is this like uploading files?" → No, OAuth integration (standard)
- "How do we know?" → Audit trail shows clone → analyze → delete
- "What if you go down?" → We delete within 1 hour (policy)

Pricing:

- **Starter**: \$299/mo, 20 repos/mo, GitHub OAuth only
- **Pro**: \$999/mo, unlimited repos, OAuth + patches
- **Custom**: \$2K+/mo, priority support

****OPTION 2: CLI Tool** ■■ (For Paranoid Devs)**

Who Uses This:

- Security-conscious teams
- "Code never leaves our machine" mandate
- DevOps engineers
- Compliance-heavy (HIPAA, SOC2)
- Budget: Any (same price as OAuth)
- Risk tolerance: High (paranoid, in good way)

How It Works:

Flow:

1. Developer: `pip install swift-cli`
2. Developer: `swift scan --repo /path/to/code --api-key xyz`
3. CLI runs LOCALLY:
 - Reads code from disk
 - Analyzes locally (Claude API calls for reasoning only)
 - Generates patches locally
 - Encrypts findings
4. Sends RESULTS to SWIFT dashboard (not code)
5. Developer sees findings in CLI + dashboard

Data Journey:

Developer Machine (your code stays here)

↓ [CLI analyzes locally, calls Claude for chain reasoning]

Claude API (brief context for reasoning, not full code)

↓ [Returns chain analysis]

Developer Machine (merges results)

↓ [Encrypts and sends findings to dashboard]

SWIFT Dashboard (results only, no code)

↓ [Developer views findings]

Implementation (Week 2-4)

```
# cli/scanner.py (enhancement)

import click
import json
from pathlib import Path
from scanners.claude_analyzer import ClaudeAnalyzer
from governance.forensic_logger import AuditLogger
from encryption.secure_results import EncryptResults

@click.command()
@click.option('--repo', type=click.Path(exists=True), required=True)
@click.option('--api-key', type=str, required=True)
@click.option('--upload', is_flag=True, default=True, help="Upload results to dashboard")
def scan(repo, api_key, upload):
    """
    Scan code LOCALLY. Optionally upload results to dashboard.
    Code NEVER leaves your machine.
    """
    click.echo(f"■ Scanning {repo} locally...")
    click.echo(" Code will not leave your machine.")

    # Initialize
    analyzer = ClaudeAnalyzer(api_key)
    audit = AuditLogger()

    # Log start (local)
    audit.log_local(f"Started scan: {repo}", method="cli")
    try:
        # Read repo (LOCAL only)
        code_content = read_repo_files(repo)
        click.echo(f" Found {len(code_content)} files")

        # Analyze locally
        vulnerabilities = analyzer.analyze_code(code_content, language="multi")
        click.echo(f" Found {len(vulnerabilities)} vulnerabilities")

        # Discover chains
        chains = analyzer.discover_chains(vulnerabilities)
        click.echo(f" Found {len(chains)} exploit chains")

        # Generate patches
        patches = analyzer.generate_patches(vulnerabilities)
        click.echo(f" Generated {len(patches)} patch candidates")
```

```

# Display results locally
display_results(vulnerabilities, chains, patches)

# Optionally upload to dashboard
if upload:
    click.echo("\n■ Uploading results to dashboard...")

    # Encrypt results
    encrypted = EncryptResults(api_key).encrypt({
        "vulnerabilities": vulnerabilities,
        "chains": chains,
        "patches": patches,
        "repo": repo,
        "timestamp": datetime.now().isoformat(),
        "method": "cli"
    })

    # Send encrypted results
    response = requests.post(
        "https://swift.app/api/results",
        json={"encrypted_results": encrypted},
        headers={"Authorization": f"Bearer {api_key}"})
    )

    click.echo(f" Results uploaded: {response.json()['dashboard_url']}")

    # Log completion (local)
    audit.log_local(f"Completed scan: {len(chains)} chains found", method="cli")
    click.echo("\n■ Scan complete. Code stayed on your machine.")

    except Exception as e:
        click.echo(f"\n■ Error: {str(e)}")
        audit.log_local(f"Scan failed: {str(e)}", method="cli", severity="error")
        raise

    def read_repo_files(repo_path: str) -> dict:
        """Read all code files from repo"""

        files = {}

        for file_path in Path(repo_path).rglob("*.py"): # Add *.js, *.go, etc
            try:
                with open(file_path, 'r') as f:
                    files[str(file_path)] = f.read()
            except:
                pass

        return files

    def display_results(vulns, chains, patches):

```

```

"""Pretty-print results to terminal"""
click.echo("\n" + "="*60)
click.echo("VULNERABILITIES")
click.echo("="*60)
for vuln in vulns:
    click.echo(f" [{vuln['severity']}] {vuln['type']}")
    click.echo(f" File: {vuln['file']}")
    click.echo(f" Line: {vuln['line']}")
    click.echo("\n" + "="*60)
click.echo("EXPLOIT CHAINS")
click.echo("="*60)
for chain in chains:
    click.echo(f" [{chain['severity']}] {chain['description']}")
    click.echo(f" Attack path: {' → '.join(chain['path'])}")
# etc...

```

Installation:

```

pip install swift-cli

swift scan --repo /path/to/code --api-key YOUR_API_KEY

# Optional: upload results to dashboard

swift scan --repo /path/to/code --api-key YOUR_API_KEY --upload

```

Client Pitch:

> "Install our CLI. Run `swift scan` locally. Your code never leaves your machine. We analyze locally, encrypt the results, and optionally send them to your dashboard. Your choice. Maximum control, maximum trust."

Concerns Addressed:

- "Code stays on our machine?" → Yes, verified in audit log
- "How do I know you're not stealing code?" → Open-source CLI, you can audit it
- "What gets sent to cloud?" → Results only, encrypted
- "Can I run offline?" → Yes, store API key locally
- "How fast?" → Instant (local execution)

Pricing:

- **Same as OAuth**: \$299-999/mo (no price difference)
- **Why**: You're paying for SWIFT's reasoning (Claude API), not for cloud storage

****OPTION 3: On-Premise Deployment** ■■■ (Enterprise)**

Who Uses This:

- Fortune 500 companies
- Healthcare (HIPAA)
- Finance (PCI-DSS)
- Government contractors
- "Code must stay in our data center" mandate
- Budget: >\$100M ARR
- Risk tolerance: Maximum paranoia (justified)

How It Works:

Flow:

```
1. Enterprise: "Deploy SWIFT on our servers"
2. SWIFT provides Docker container
3. Enterprise deploys in their data center
4. SWIFT runs inside their network:
- Reads code from internal Git servers
- Analyzes locally
- Stores results on their infrastructure
- No data leaves network
5. Optional: SWIFT cloud dashboard shows metadata only (no code/vulns)

Data Journey:
Enterprise Git Server (your code)
↓ [SWIFT pulls from internal Git]
SWIFT Container (on your servers)
↓ [Analyzes, stores results]
Your infrastructure (results stay here)
↓ [Optional: send metadata to SWIFT cloud dashboard]
SWIFT Cloud (metadata only: "analyzed on 2024-04-20, found 5 chains")
```

Implementation (Week 2.0)

```
# docker/Dockerfile.onprem

FROM python:3.11-slim

WORKDIR /app

# Copy SWIFT code
COPY swift/ /app/swift/
COPY requirements.txt /app/

# Install dependencies
RUN pip install -r requirements.txt

# Copy startup script
COPY docker/entrypoint.sh /app/

# Configuration
ENV SWIFT_MODE=onprem
ENV ANTHROPIC_API_KEY=YOUR_KEY_HERE
ENV INTERNAL_GIT_URL=https://internal-git.company.com
ENV STORAGE_PATH=/data/swift_results

# Expose API
EXPOSE 8000

ENTRYPOINT ["/app/entrypoint.sh"]
```

```
# docker/entrypoint.sh (adapted)

#!/bin/bash
```

```
# On-prem mode:

# - Read from internal Git only

# - Store results locally

# - Never contact SWIFT cloud (optional)

# - Encrypt results locally

python /app/swift/api/onprem_server.py \

--mode=onprem \

--git-url=$INTERNAL_GIT_URL \

--storage=$STORAGE_PATH \

--cloud-sync=disabled # No data leaves network
```

Deployment:

```
# Enterprise deploys SWIFT on their infrastructure

docker run \

-v /company/git:/internal_git \

-v /company/swift_data:/data \

-e ANTHROPIC_API_KEY=YOUR_KEY \

-e INTERNAL_GIT_URL=file:///internal_git \

-p 8000:8000 \

swift:onprem

# Access at: https://internal-swift.company.com:8000

# Results stored in: /company/swift_data/

# Code never leaves network
```

Client Pitch:

> "We provide the SWIFT Docker container. You deploy it in your data center. Code stays on your network. Results stay on your network. Optional: send anonymized metadata to our cloud dashboard for visibility across your deployments. Zero data leaves your infrastructure."

Concerns Addressed:

- "Does code leave our network?" → No. Verified by network audit
- "Can you access our data?" → No. SWIFT runs inside firewall only
- "What about compliance (HIPAA/PCI)?" → Enterprise can certify themselves
- "Long-term support?" → Included in contract
- "Can we audit the code?" → Yes, provided in deployment

Pricing:

- **Base**: \$15K/year (Docker image license)
- **Support**: \$5K-20K/year (depends on SLA)
- **Cloud sync**: Optional \$2K/year (metadata dashboard)
- **Total**: \$20K-35K/year typical enterprise

The Product Strategy (Adaptive)

Access Method Decision Tree:

```
CLIENT ARRIVES
```

↓

Q1: "How risk-averse are you?"

- ■ Low risk (startup) → GitHub OAuth
- ■ Medium risk (mid-market) → CLI Tool (for devs) or OAuth (for operations)
- ■ High risk (enterprise) → On-Prem or Air-gapped

↓

Q2: "Where is your code?"

- ■ GitHub/GitLab → OAuth (easy)
- ■ Local machines → CLI (local-first)
- ■ Internal servers → On-Prem (isolated)

↓

Q3: "What's your compliance requirement?"

- ■ None → OAuth (cheapest, fastest)
- ■ SOC2/ISO → OAuth + audit logs
- ■ HIPAA/PCI → On-Prem (required)
- ■ Air-gapped → Offline CLI + manual upload

↓

SWIFT ADAPTS:

- OAuth → Fast onboarding, SaaS model
- CLI → Local-first, results-only upload
- On-Prem → Enterprise support, isolated deployment

Implementation Timeline

Week 1-2: Core (exploit chain discovery) ← Public repos

Week 3: Add GitHub OAuth integration

- ■ GitHubIntegration class
- ■ OAuth token handling
- ■ Test on public repos

Week 3-4: Add CLI Tool (parallel)

- ■ enhance cli/scanner.py
- ■ Local analysis
- ■ Encryption for results
- ■ Test locally

Week 5-6: Customer validation

- ■ "Which access method?"
- ■ GitHub OAuth: startups
- ■ CLI: paranoid devs

■ ■ Collect feedback

Week 7-8: Polish both

■ ■ OAuth: smooth onboarding

■ ■ CLI: easy installation

■ ■ Documentation

Week 9-10: Launch both

■ ■ "Choose your access method"

■ ■ GitHub OAuth (default)

■ ■ CLI Tool (alternative)

■ ■ Public messaging

Week 11-12: Close customers

■ ■ Startups: OAuth

■ ■ Paranoid devs: CLI

■ ■ Enterprise: start on-prem conversation

Week 13+: On-prem (premium)

■ ■ Docker packaging

■ ■ Enterprise support

■ ■ Higher pricing

Updated Pricing Model

Sales Pitch (Adaptive)

For Startup Founder:

> "Choose how you're comfortable. GitHub OAuth is easiest—2 minutes to connect. We'll scan your repos, delete the code, show you chains. Like GitHub Advanced Security. \$299/mo to start."

For Security-Conscious Dev:

> "Install our CLI. Runs locally on your machine. Code never leaves. Optional upload. \$299/mo same price, your choice."

For Enterprise CISO:

> "Deploy SWIFT in your data center. Docker container. Code stays on your network. Full compliance. We support from outside your firewall. \$25K/year."

Why This Works

1. No client rejected — Everyone finds their comfort level
2. Same product — All methods use same Claude analysis
3. Same price (mostly) — You're not charging for storage, you're charging for reasoning
4. Different trust models — Clients choose what matters to them
5. Easy upsell — Start OAuth → want isolation? Switch to CLI → need compliance? Upgrade to on-prem

Competitive Advantage

You're the only one offering all three from day one.

Updated Execution Plan Week 3

Week 3 Checklist:

GitHub OAuth:

[] Create GitHubIntegration class

[] Implement OAuth flow

[] Add token refresh logic

[] Test on 3 public repos

[] Build GitHub auth UI component

CLI Enhancement:

[] Enhance cli/scanner.py with local analysis

[] Add encryption for results

[] Implement optional dashboard upload

[] Build installation package (pip)

[] Test on 3 repos locally

Both:

[] Create access method selector UI

[] Document both flows

[] Set up environment variables

[] Add to audit logging (which method used?)

[] Test both flows end-to-end

On-prem (prep for Week 8):

[] Create docker/Dockerfile.onprem

[] Plan internal Git integration

[] Design network isolation

[] Document deployment guide

Bottom Line

You don't offer ONE access method. You offer THREE.

Clients ask: "How do you handle code?"

You answer: "Choose your comfort level:"

- GitHub OAuth? Done in 2 minutes
- CLI locally? Code stays on your machine
- On-prem? Container on your servers
- All the same price (mostly)
- All the same power (exploit chains)

That's the product. That's the moat.

Everyone else: "Upload your code or OAuth"

SWIFT: "Choose."

Questions:

1. Should OAuth be default (easier), or CLI as default (safer)?
2. What's your timeline for on-prem? Week 8 or Week 11?
3. Should CLI be paid extra or same price?

Competitive Analysis: Code Access Problem

How Competitors Actually Solve This

1. VERACODE (Enterprise Standard)

Model: SaaS + On-Premise + API

Access Methods:

Option A: Web Upload

- Developer uploads ZIP file
- Veracode scans on their servers
- Results in 2-7 days

Option B: API Integration (GitHub/GitLab/Bitbucket)

- OAuth: "Veracode, scan my GitHub repo"
- Automatic: Every commit triggers scan
- Results in hours (CI/CD)

Option C: On-Premise (Enterprise)

- "Deploy on your servers"
- Code NEVER leaves company network
- Highest trust/compliance tier
- Cost: \$500K+/year

Why it works:

- Enterprise buying = Legal teams negotiate
- "It's not uploading, it's OAuth integration"
- On-prem option for ultra-paranoid companies

Revenue Model:

- Startup: \$50K-100K/year (SaaS upload)
- Mid-market: \$200K-300K/year (CI/CD integration)
- Enterprise: \$500K-1M/year (on-prem)

2. SNYK (Developer-First)

Model: SaaS + GitHub integration + CLI

Access Methods:

Option A: GitHub/GitLab OAuth

- "Connect your repo"
- Automatic scanning on push
- Code stays on GitHub (Snyk pulls dependencies)
- NO FILE UPLOAD

Option B: CLI Tool (Local)

- `npm install -g snyk`
- `snyk test` (runs locally, results sent to cloud)
- Code STAYS on dev machine

- Upload = results only, not code
- Option C: Dependency Database
- Snyk scans: package.json, requirements.txt, go.mod
 - NOT the actual source code
 - Just dependency versions vs CVE database

Why it's genius:

- **Never uploads actual source code**
- Only scans dependencies (public data)
- Developers LOVE the CLI (local-first mindset)
- GitHub integration = zero friction

Revenue Model:

- Free tier: 5 projects
- Paid: \$2K-5K/year (unlimited projects)
- Enterprise: \$50K+/year

3. GITHUB ADVANCED SECURITY (Cloud Native)

Model: GitHub-Native (code stays on GitHub)

Access Method:

Option A: CodeQL (Built-in to GitHub)

- No upload needed
- Runs on GitHub's servers
- Scans your repo that already lives there
- Results show in PR

Option B: Secret Scanning

- Scans commits for API keys
- Real-time on GitHub

Option C: Dependabot

- Checks dependencies vs CVE DB
- Opens pull requests with fixes

Why it works:

- **Code already on GitHub** = implicit trust
- No "upload" needed, just authorization
- Friction: ZERO
- Cost: \$21/user/month (included in GitHub)

4. SEMGREP (Developer Community)

Model: Open-source CLI + Cloud (Optional)

Access Methods:

Option A: Local CLI (No Upload)

- `semgrep --config=p/security-audit .``
- Runs entirely on dev machine
- Results = text output only
- Code NEVER leaves local machine

```
Option B: Cloud Dashboard (Optional)

- `semgrep login`

- Upload = RESULTS ONLY (findings, not code)

- No source code stored

- Just for visibility/tracking

Option C: CI/CD Integration

- Runs in GitHub Actions/GitLab CI

- Scans code on YOUR infrastructure

- Only results uploaded
```

Why developers love it:

- **Trust by design** (runs local first)
- Transparent open-source
- No mandatory cloud
- Cost: Free tier is powerful

5. FORTIFY (HP/Micro Focus, Legacy)

Model: Enterprise SaaS + On-Premise

Access Method:

```
Option A: Web Upload

- ZIP file upload (painful)

- Scan takes days

- Expensive

Option B: On-Premise Server

- Deploy on your network

- Code never leaves

- Old architecture = slow
```

Problem: Slow, expensive, enterprise-only GTM

The Pattern (Why They Win)

Key insight: Nobody is asking startups to "upload your code." They either:

1. Integrate with GitHub/GitLab (code already lives there)
2. Provide CLI (code runs locally, results uploaded)
3. Deploy on-prem (code never leaves customer servers)
4. Scan dependencies only (public data, no code needed)

Why SWIFT (You) Has an Advantage Here

Your competitors scan:

- Snyk: Dependencies only (fast, limited)
- Veracode: Individual bugs (slow, misses chains)
- GitHub: Pattern matching (cheap, shallow)

You scan:

- Exploit chains (deeper reasoning needed)
- Requires actual code analysis (not just dependencies)

Problem: You NEED actual source code to find chains

But: Startups won't upload it

Solution: You have 3 options (like everyone else)

OPTION 1: GitHub Integration (OAuth) — Recommended

How It Works:

```
User clicks: "Connect GitHub"
↓
OAuth prompt: "SWIFT wants to read your repos"
↓
User approves
↓
SWIFT clones repo from GitHub to your server
↓
Run analysis (chain discovery)
↓
Delete repo from your server
↓
Send results back to user
```

Why it works:

- ■ Code never stays on your servers (delete after scan)
- ■ No "upload" feeling (it's OAuth integration)
- ■ Automatic (CI/CD trigger on push)
- ■ Developer-friendly (how GitHub Advanced Security does it)

Implementation:

```
# Week 3-4 addition:

class GitHubIntegration:

    def get_repo_via_oauth(self, user_token, repo_name):

        # Use GitHub API + user's token

        # Clone repo to temp directory

        response = requests.get(

            f"https://api.github.com/repos/{repo_name}/contents",
```

```

headers={"Authorization": f"token {user_token}"}
)

return response

def cleanup_after_scan(self, temp_dir):
    # Delete repo after analysis
    shutil.rmtree(temp_dir)

```

Startup concerns addressed:

- "Your code never stays on our servers" ✓
- "We use GitHub's OAuth, not sketchy uploads" ✓
- "You approve each scan" ✓
- "Code is deleted after analysis" ✓

OPTION 2: CLI Tool (Local First) — Developer Friendly

How It Works:

```

Developer: `pip install swift-scanner`

`swift scan --repo . --api-key xyz`

↓

CLI runs analysis locally (on their machine)

↓

Results sent to SWIFT dashboard (encrypted)

↓

Dashboard shows: "Here are your chains"

```

Why it works:

- ■ Code NEVER leaves developer machine
- ■ Maximum trust (Semgrep model)
- ■ Fast feedback (local execution)
- ■ Works offline

Implementation:

```

# cli/main.py (already exists, just enhance it)

@click.command()

@click.option('--repo', type=click.Path(exists=True))

@click.option('--api-key', type=str)

def scan(repo, api_key):

    analyzer = ClaudeAnalyzer(api_key)

    # Scan LOCALLY

    vulnerabilities = analyzer.analyze_code_local(repo)

    chains = analyzer.discover_chains(vulnerabilities)

    # Send RESULTS to cloud (not code)

```

```
send_to_dashboard(api_key, chains)

# Display locally

print_findings(chains)
```

Startup concerns addressed:

- "Code never leaves our machine" ✓✓✓
- "We can audit the code (open-source CLI)" ✓
- "Instant feedback (no waiting for cloud)" ✓

OPTION 3: On-Premise Deployment — Enterprise Trust

How It Works:

```
Enterprise says: "Deploy SWIFT on our servers"
↓
You provide Docker container
↓
They deploy in their data center
↓
SWIFT runs inside their network
↓
Code never leaves their network
↓
Results stay in their infrastructure
```

Why it works:

- ■ Absolute maximum trust
- ■ Compliance-friendly (HIPAA, SOC2, etc)
- ■ Premium pricing (\$5K-50K/mo)
- ■ How Veracode sells to Fortune 500

Implementation:

```
# docker/Dockerfile

FROM python:3.11

COPY swift/ /app/swift/

COPY docker/start.sh /app/

CMD ["/app/start.sh"]

# Customer runs:

# docker run -v /their/code:/code swift-scanner /code
```

OPTION 4: Hybrid: Public Repos (Freemium) + Private OAuth/On-Prem (Paid)

Freemium Tier (No code sharing needed):

User uploads public GitHub repo URL

↓

We scan it (public code anyway)

↓

Show findings

↓

Free demo

Paid Tier:

User chooses:

- OAuth: "Connect your private GitHub repo"
- CLI: "Scan locally, upload results"
- On-prem: "Deploy SWIFT in our infrastructure"

How KALI LINUX Fits In

You mentioned Kali Linux + security tools. Let me clarify:

What Kali provides:

- **Penetration testing tools** (Metasploit, Burp, etc)
- **Network scanning** (Nmap, Wireshark)
- **Post-exploitation** (reverse shells, privilege escalation)
- **System-level security** (hash cracking, wireless pentesting)

What SWIFT provides:

- **Code-level analysis** (find bugs before they're exploited)
- **Static analysis** (no running code needed)
- **Exploit chain reasoning** (AI-powered, not manual)
- **Automation** (CI/CD, not one-off assessments)

They're complementary:

Developer workflow:

1. SWIFT scans code → finds chains + patches
2. Patches deployed to staging
3. Kali/pentester breaks staging (validates patches work)
4. Patches deployed to production
5. Repeat on next commit

KALI doesn't solve code upload problem because:

- It's a toolkit (you still need to run it manually)
- It's offensive (assumes you're already in the system)
- It's not continuous (annual pentests, not per-commit)

RECOMMENDED APPROACH FOR SWIFT

Phase 1 (Weeks 1-6): Prove Concept

Start with: PUBLIC repos only

- No code sharing needed
- Publish findings (case studies)
- Prove chain discovery works

Phase 2 (Weeks 7-10): Launch

Offer: OAuth GitHub integration OR Freemium

- "Connect GitHub repo (private or public)"
- "Code scanned on our servers, deleted after"
- "Results shown in dashboard"

Phase 3 (Weeks 11-12+): Expand

Add: CLI tool OR On-prem option

- For security-conscious startups
- For enterprises
- Premium pricing

Updated Competitive Position

Your Positioning

"We find exploit chains (not just bugs) via GitHub OAuth (no friction, code deleted after)"

- Like Snyk's ease (GitHub native)
- Like Veracode's power (deep analysis)
- Like GitHub's integration (zero friction)
- Unique: Exploit chain reasoning

Implementation Timeline

Week 1-2: Public repos (no access needed)

Week 3-4: Add GitHub OAuth integration

Week 5-6: Validate with startups (they'll say "yes" to OAuth)

Week 7-8: Add CLI tool (for paranoid developers)

Week 9-10: Launch (GitHub OAuth + freemium public repos)

Week 11-12: Offer on-prem option (enterprise customers)

Bottom Line

You don't need "code upload" at all.

- **Snyk model**: Scan dependencies (not full code) → Works, but you need full code
- **GitHub model**: OAuth integration → Code comes from GitHub, analyzed, deleted
- **Semgrep model**: CLI tool → Code stays local, results uploaded
- **Veracode model**: On-prem deployment → Code stays in customer network

SWIFT should use GitHub OAuth + CLI as primary, on-prem as premium.

Problem solved. No startup paranoia. No "upload" friction.

Next question: Should I update the execution plan to include GitHub OAuth integration by Week 3-4?

SWIFT: IP Protection & Security Strategy

"Protecting Your Moat While Serving Clients"

The Risk (Real)

What You're Worried About:

```
You: Build SWIFT (exploit chain discovery)

Client: Gets access to your code

Client: "Hey, this is good. Let's build our own version"

Client: Sells competing product using your code

You: Lose market advantage
```

Is This Real?

YES, it happens. Examples:

- **Stripe competitors**: Copied checkout flow
- **Slack clones**: Built similar products with Slack's API
- **Vercel competitors**: Copied Next.js deployment strategy
- **GitHub Copilot controversy**: AI trained on public code

What Makes SWIFT Special?

Your moat is:

1. Exploit chain reasoning (Claude 3.5 Sonnet can do this)
2. Governance layer (auditing, permissions, approval workflow)
3. Patch generation + validation (end-to-end automation)

If clients steal your code, they get:

- Python codebase (public, easy to read)
- API calls to Claude (anyone can do this)
- Governance logic (copyable, not patented)

Reality: Someone WILL eventually build a competing product. The question is: Can you stay ahead?

The REAL Answer: You Don't Share Your Code

Let me reframe:

■ WRONG: "We share SWIFT source code with clients"

```
Client gets:

- Your entire codebase
- Architecture
- Prompt strategies
- Governance logic
- Patches + exploits

→ They can build their own version in weeks

→ You lose competitive advantage
```

■ RIGHT: "We share SWIFT service, not source code"

Client gets:

- GitHub OAuth API (your endpoint)
- CLI binary (compiled, not source)
- Results (findings, patches, audit logs)
- Dashboard (SaaS interface)

Client DOES NOT get:

- Your Python code
- Claude prompts
- Chain discovery logic
- Governance internals
- Patch generation strategy

This is the standard SaaS model. Snyk doesn't share source. GitHub doesn't. Veracode doesn't.

What You Actually Share (Safe Things)

■ SAFE TO SHARE: Client Sees Results

WHAT THEY SEE (Results):

```
{
  "vulnerabilities": [
    {
      "type": "SQL Injection",
      "file": "app/models/user.py",
      "line": 42,
      "severity": "HIGH",
      "description": "User input not parameterized"
    }
  ],
  "chains": [
    {
      "chain_id": "CHAIN_001",
      "description": "SQL injection + authentication bypass = RCE",
      "severity": "CRITICAL",
      "attack_path": ["SQL inject", "bypass auth", "escalate"]
    }
  ],
}
```

```
"patches": [  
  {  
    "vuln_id": "SQL_001",  
    "patch": "Use parameterized queries",  
    "code_diff": "..."  
  }  
],  
"audit_trail": [  
  {  
    "action": "Analyzed repo",  
    "timestamp": "2026-04-17T10:00:00Z",  
    "user": "alice@startup.com",  
    "hash": "HMAC_SIGNATURE"  
  }  
]  
}
```

■ NEVER SHARE: Your Secret Sauce

Your Code (PROTECTED)

```
# scanners/claude_analyzer.py ← NEVER SHARE  
  
class ClaudeAnalyzer:  
    def discover_chains(self, vulns):  
        prompt = """  
        You are a security expert analyzing vulnerabilities.  
        For these vulnerabilities: [LIST]  
        Find chains where multiple vulns combine.  
        Key insight: Look for:  
        1. Data flow dependencies  
        2. State transitions  
        3. Permission cascades  
        4. Timing windows  
        Return only feasible chains.  
        """  
  
        # This prompt is your IP
```

Why? Anyone who sees this prompt can:

- Use it with their own Claude API key
- Build competing product in days

- Sell it as their own

Your Governance Logic (PROTECTED)

```
# governance/permission_engine.py ← NEVER SHARE

class PermissionEngine:

def check_access(self, user, repo, action):

# Your business logic for who can scan what

# Your audit log strategy

# Your approval workflow

# Your escalation procedures
```

Your Patch Generation Strategy (PROTECTED)

```
# remediation/patch_generator.py ← NEVER SHARE

class PatchGenerator:

def generate_patches(self, vuln):

prompt = """

Generate 3-5 patch candidates for this vulnerability.

Rank by:

1. Maintains backward compatibility
2. No performance regression
3. Minimal code change
4. Production-safe

"""
```

What You DO Share: The Service

Architecture Clients See:

```
Client Dashboard (Browser):

- Input: "Connect GitHub" or "Upload code"
- Output: "Vulnerabilities + Chains + Patches"
- No code visible

CLI Tool (Binary/Package):

- Client runs: `swift scan --repo . --api-key xyz`
- Binary (not source code)
- Sends code to your servers (encrypted)
- Returns results only

API (REST):

- Endpoint: POST /api/scan
```

- Input: repo_url or code
- Output: findings JSON
- Client never sees implementation

What clients can reverse-engineer:

- API request/response format (not useful)
- Which vulnerabilities you detect (they'll learn anyway)
- That you use Claude (obvious from cost/latency)

What clients CAN'T reverse-engineer:

- Your prompts
- Your chain discovery logic
- Your patch generation strategy
- Your governance logic
- Your audit log architecture

Security Model (Smart)

Option 1: GitHub OAuth (Recommended for Startups)

```
Client: "Connect GitHub"
↓
OAuth: Grants SWIFT permission to read repos
↓
YOUR SERVERS:
- Clone repo (in container)
- Read code (on your infrastructure)
- Analyze (your prompts, your logic)
- Delete code (after 1 hour)
- Send results back
↓
Client: Sees results only
Client: Never sees your code
Client: Never sees your prompts
Client: Never sees your logic
```

Client can see:

- API responses (findings)
- Dashboard (results)

Client CANNOT see:

- Your Python code
- Your Claude prompts
- Your analysis logic

Option 2: CLI Tool (Local Analysis, Safer)

```
Client: `swift scan --repo .`
```

↓

CLI (binary, not source):

- Reads local code
- Calls YOUR Claude API (from your account)
- You see the analysis request
- Returns results

↓

Client: Sees results

Client: Doesn't see your code OR your prompts

Why safer?

- Code analyzed on CLIENT's machine (you never see it)
- But they use YOUR Claude API key calls
- Prevents them from using their own Claude with your prompts
- You can audit usage

Option 3: On-Prem (Enterprise, Most Secure)

Client: Deploys Docker container (on their servers)

↓

Container runs inside their network:

- Code on their machines (you never see it)
- Analysis happens on their infrastructure
- Results stay local
- Optional: send metadata to your dashboard

↓

You see: Metadata only ("scan completed, 5 chains found")

You see: NOT the actual code or vulnerabilities

Client: Maximum security

Why secure?

- Code never leaves their network
- Your binary runs, but not your source
- You get zero information about findings

Protecting Your IP (Legal + Technical)

1. Terms of Service (Mandatory)

SWIFT TERMS OF SERVICE

Section 3: Intellectual Property

3.1 License Grant

You are granted a limited, non-exclusive, non-transferable

license to use SWIFT for vulnerability scanning.

3.2 Restrictions

You shall NOT:

- Decompile, reverse-engineer, or disassemble SWIFT
- Extract or copy SWIFT's source code
- Use SWIFT to train competing products
- Share SWIFT credentials with third parties
- Run SWIFT's analysis on behalf of other companies

3.3 Penalties

- Breach: Immediate termination + \$10,000 liquidated damages
- IP theft: Up to \$50,000 per violation

3.4 Data Ownership

- You own your code
- SWIFT owns all analysis logic, prompts, and governance
- You own the findings (vulnerabilities discovered)
- You own audit logs

3.5 Audit Rights

- We reserve the right to audit your usage
- Detect unauthorized reverse-engineering

2. Technology Protection (API Design)

```
# api/routes.py

@app.post("/api/scan")

def scan_endpoint(request: ScanRequest, token: str = Header(...)):
    """
    Scan endpoint - client never sees internals
    """

    # 1. Validate token
    if not validate_api_token(token):
        return {"error": "Unauthorized"}

    # 2. Rate limiting (prevent abuse/analysis)
    if exceed_rate_limit(token):
        return {"error": "Rate limit exceeded"}

    # 3. Accept code (never send back)
    code = request.code or clone_from_github(request.repo_url)

    # 4. Analyze ON YOUR SERVERS (client can't see this)
    findings = analyze_code_internally(code) # ← Your secret logic

    # 5. Return ONLY results (not internals)
    return {
```

```
"vulnerabilities": findings["vulnerabilities"],
"chains": findings["chains"],
"patches": findings["patches"]
# NOT included: prompts, logic, reasoning chains
}
```

3. Code Obfuscation (Binary Release)

```
# For CLI tool distribution:
# Your Python code (protected):
# scanners/claude_analyzer.py (secret)
# governance/permission_engine.py (secret)
# remediation/patch_generator.py (secret)
# Compiled for distribution:
# swift-cli (PyInstaller binary - not readable)
# Client gets: Binary executable, not source
pyinstaller --onefile cli/main.py # Creates swift-cli (binary)
# Client can't reverse-engineer easily
```

4. API Key Management

```
# Your Claude API key (NEVER shared)
# Client's SWIFT API key (for authentication only)
class APIKeyManager:
    def __init__(self):
        self.claude_api_key = os.environ["ANTHROPIC_API_KEY"] # YOUR key
        self.client_keys = {} # Client keys for auth only
    def scan_with_client_key(self, client_key, code):
        """
        Client authenticates with their SWIFT API key
        But analysis uses YOUR Claude key
        This prevents clients from using their own Claude
        with your prompts
        """
        if not validate_client_key(client_key):
            raise Unauthorized()
        # Analysis uses YOUR key
        result = ClaudeAnalyzer(self.claude_api_key).analyze(code)
        return result
```

Why? If clients had their own Claude key, they could:

- Call Claude API themselves
- Use your prompts (if they reverse-engineered them)
- Build competing product

By using YOUR Claude key for analysis, you:

- Control all analysis (they can't replicate without you)
- Can audit Claude token usage
- Force them to stay on your platform

Realistic Risk Assessment

Will Clients Copy SWIFT?

Probability: MEDIUM (60-70%)

Why?

- Your code will eventually leak (disgruntled employee, breach, etc)
- Exploit chain discovery is replicable
- Claude API is public

When?

- **Month 0-3**: Unlikely (you're moving fast, they're not)
- **Month 4-6**: Possible (someone smart figures it out)
- **Month 6-12**: Likely (competitors wake up)
- **Year 2**: Inevitable (someone builds clone)

What's Your Defense?

1. Speed: Move faster than competitors can copy
 - Weeks 1-12: Get to market, acquire customers
 - Month 4-6: Have 20+ paying customers, revenue, brand
 - When copy happens: You're already market leader
2. Trust: Build relationship moat
 - Governance layer becomes trusted standard
 - Audit logs become regulatory requirement
 - Customers lock in to your system (switching cost)
 - Patches are production-tested (copying code ≠ copying expertise)
3. Features: Stay ahead
 - Week 12: Chain discovery + patches + governance
 - Month 4: Add X feature competitors don't have
 - Month 8: Add Y feature competitors can't copy fast
 - Keep innovating (they're always catching up)
4. Legal: Make copying expensive
 - TOS with liquidated damages (\$10K per violation)
 - Patent key methods (if possible)
 - Trademark "SWIFT" brand
 - Copyright on code + prompts

Is This a Problem?

NO, for these reasons:

1. First-mover advantage (you're months ahead)
2. Customer lock-in (audit logs, governance = switching cost)
3. Expert moat (you understand the system better)
4. Trust moat (you're the original, copy is the knockoff)
5. Market timing (in 12 months, the market explodes, room for 2-3 winners)

Example: GitHub had 50+ competitors. Still market leader. Why?

- First to market
- Better UX
- Better trust
- Better features
- Better brand
- Fast execution

Same here.

What You Actually Fear (Real Fear)

Fear #1: "Big company will just copy us"

Risk: Google/Microsoft/Amazon builds SWIFT clone

Reality:

- They're slow (12-24 month cycle)
- You'll have 100+ customers + revenue by then
- You'll either be acquired or scaled
- Competition doesn't kill you, it validates market

Fear #2: "Client will steal our code and compete"

Risk: Startup client gets your code, builds their own

Reality:

- Your TOS forbids it (\$10K damages if caught)
- They'd need to build entire product (hard)
- You're moving fast, they're not
- By the time they clone it, you've added 10 features
- If they do, you can pursue legal action

Fear #3: "What if they modify SWIFT for internal use?"

Risk: Client uses SWIFT internally to scan their code base

Reality:

- This is ALLOWED (they're not competing)
- They pay you (revenue)
- You win
- This is your business model

Your Protection Strategy (Practical)

SHORT TERM (Weeks 1-12: MVP Launch)

■ DO:

- Don't share your source code
- Give clients access to SERVICE (API/CLI)
- Use GitHub OAuth (code never sits on your servers)
- Use CLI binary (not source)
- Strong TOS with IP protection clause
- NEVER:
- Don't share scanners/claude_analyzer.py
- Don't share governance/permission_engine.py
- Don't share remediation/patch_generator.py
- Don't expose your Claude prompts
- Don't document your chain discovery algorithm (yet)
- DOCUMENT:
- API specification (client sees)
- Results format (client sees)
- Setup guide (client sees)
- NOT: internal logic, prompts, governance internals

MID TERM (Months 4-12: Scaling)

- DO:
- Publish blog post about exploit chains (intellectual credit)
- Give talks at conferences (build brand/moat)
- Patent key methods (if applicable)
- Build strong customer relationships (switching cost)
- Add governance features competitors can't copy fast
- Publish governance standard (become the standard)
- PREPARE:
- Detect reverse-engineering attempts (audit logs)
- Monitor for code leaks (use LSASS-like detection)
- Track who accesses what (TOS violation detection)
- Be ready to sue if needed (lawyer on retainer)

LONG TERM (Year 2+: Market Leader)

- DO:
- Open source non-core components (governance layer)
- Build community (contributors = lock-in)
- Become the standard (enterprise requirement)
- Expand into adjacent markets
- Patent protection expires, but you're too far ahead

TL;DR: How to Protect SWIFT

■ DON'T:

- Share your Python source code
- Expose your Claude prompts
- Publish your chain discovery algorithm
- Give clients direct access to your infrastructure

■ DO:

- Share the SERVICE (API, CLI binary, dashboard)
- Protect your IP (TOS + legal)
- Move fast (ahead of competitors)
- Build trust (customer lock-in)
- Document publicly (build brand)

ANALOGY:

You wouldn't give clients access to Snyk's source code.

They just use the SERVICE.

Same with SWIFT.

Clients see:

- "Please scan my code"
- "Here are your findings"
- "Here's your audit trail"

Clients DON'T see:

- How you discover chains
- What prompts you use
- How you generate patches
- How governance works (internals)

Final Answer to Your Question

"Is it safe to share our code with the client?"

NO. Don't.

INSTEAD: Share the service.

Client gets access to:

- ✓ API endpoint
- ✓ CLI tool (binary)
- ✓ Dashboard (web UI)
- ✓ Results (findings + patches + audit logs)

Client does NOT get:

X Your source code
X Your prompts
X Your logic
X Your governance internals

Result:

- Client can't copy your product
- Client can't reverse-engineer your prompts
- Client stays on your platform (dependent)
- You stay ahead (innovating)
- You win

Confidence Check

- ■ Your code is protected
- ■ Your prompts are protected
- ■ Your moat is real (not source code, but expertise + speed)
- ■ Competitors WILL eventually emerge (normal)
- ■ But you'll be 12-24 months ahead (enough to win)
- ■ Legal protection adds enforcement layer

You're secure. Build with confidence.

Document created: April 17, 2026

Next checkpoint: Week 3 (API design security review)

SWIFT: Reality Check

Part-Time (20-30 hrs/week) vs Full-Time

ORIGINAL PLAN vs YOUR REALITY

WHAT STAYS THE SAME

- Core idea (exploit chains) — SAME
- Claude + Haiku/Sonnet strategy — SAME
- GitHub OAuth + CLI access model — SAME
- IP protection strategy — SAME
- Pricing (\$299-999/mo) — SAME
- Target market (startups first) — SAME

WHAT CHANGES

■ Remove (Too Complex for Part-Time MVP)

1. Full GitHub OAuth UI
 - ■ Takes 20+ hours (authentication flow, token refresh, rate limiting)
 - ■ Instead: Simple API calls, manual GitHub token auth
 - When: Add full UI in Week 15+ (if needed)
2. On-Prem Deployment
 - ■ Takes 15+ hours (Docker, multi-environment, docs)
 - ■ Instead: SaaS only for MVP
 - When: Add on-prem option in Month 4+ (enterprise feature)
3. Advanced Governance Layer
 - ■ Takes 20+ hours (RBAC, audit logs, encryption, behavior monitoring)
 - ■ Instead: Simple audit logs (who scanned what, when)
 - When: Enhance in Week 15+
4. Docker Sandbox for Patch Testing
 - ■ Takes 15+ hours (Docker API, test orchestration, validation)
 - ■ Instead: Generate patches, manual validation (human approval anyway)
 - When: Add automated sandbox in Week 20+
5. Full Web Dashboard
 - ■ Takes 20+ hours (frontend, auth, data visualization)
 - ■ Instead: CLI-first + email reports
 - When: Add dashboard in Month 3+

■ Keep (MVP Essentials)

1. Exploit Chain Discovery

- ■ 20-25 hours (core logic, this is where the value is)
 - Non-negotiable
2. Haiku + Sonnet Analysis
 - ■ 15 hours (pattern matching + deep analysis)
 - Essential for differentiation
 3. CLI Tool
 - ■ 10 hours (simple, elegant, developer-friendly)
 - Easier than OAuth, works just as well for MVP
 4. Patch Generation (Basic)
 - ■ 15 hours (Claude generates diffs, manual approval)
 - Valuable, but not polished
 5. Case Studies
 - ■ 5 hours (document findings on public repos)
 - Your credibility + sales collateral
 6. Landing Page
 - ■ 5 hours (simple Webflow or HTML)
 - Looks professional, minimal effort
 7. Basic Audit Logs
 - ■ 3 hours (JSON file + timestamps)
 - Good enough for MVP

REVISED PHASE BREAKDOWN

Phase 1: Core (Weeks 1-4, ~100 hrs) ← KEY PHASE

What: Get exploit chains working

Why: This is your differentiator

Deliverable: Working analyzer on public repos

Week 1: API setup + environment (6 hrs actual work)

Week 2: Haiku triage (10 hrs actual work)

Week 3: Sonnet analysis (15 hrs actual work)

Week 4: Chain discovery (20 hrs actual work)

Week 1-4 BUFFER: Troubleshooting (20-30 hrs)

Total: ~80-100 hrs

Cost: \$30

Go/No-Go: If chains aren't working by Week 4 → re-evaluate

Phase 2: Validation (Weeks 5-8, ~80 hrs)

What: Talk to customers, get feedback

Why: Validate market demand

Deliverable: 3+ interested customers, feedback document

Week 5-6: Customer outreach (20 hrs)

- Email 10 prospects
- Conduct 3-5 calls
- Take notes

Week 7-8: Polish + iterate (30 hrs)

- Improve chain discovery based on feedback
- Create landing page
- Write case studies
- Set up Stripe

Week 5-8 BUFFER: Miscellaneous (20-30 hrs)

Total: ~70-80 hrs

Cost: \$50-100

Go/No-Go: If 0 customers interested → pivot or kill

Phase 3: MVP Launch (Weeks 9-12, ~70 hrs)

What: Get to market, get first customers

Why: Prove business model

Deliverable: 1-2 paying customers, \$300-999/mo revenue

Week 9: CLI tool + GitHub integration (simple) (15 hrs)

Week 10: Testing + polish (15 hrs)

Week 11: HN launch (10 hrs)

Week 12: Sales + onboarding (15 hrs)

Week 9-12 BUFFER: Surprises (15-20 hrs)

Total: ~70 hrs

Cost: \$200

Go/No-Go: If 0 customers → re-assess pricing or positioning

Phase 4: Growth (Weeks 13-20, ~120 hrs)

What: Add features, get 2-3 more customers

Why: Hit \$2-5K MRR, validate scalability

Deliverable: 3-5 customers, \$2-5K MRR

Week 13-16: Feature development (40 hrs)

- GitHub OAuth full UI (if needed)
- Better error handling
- Customer-requested features

Week 17-20: Sales + customer success (40 hrs)

- Direct outreach
- Customer onboarding
- Support

Week 13-20 BUFFER: Miscellaneous (20-40 hrs)

Total: ~100-120 hrs

Cost: \$400-600

Go/No-Go: If not hitting \$1K+ MRR → reassess

TOTAL TIME & BUDGET

Weeks 1-20: ~320-360 hours (about 6-7 hours/week average)

Budget: ~\$800-1000 (mostly API calls)

Breakdown:

- Phase 1 (Core): 80-100 hrs, \$30
- Phase 2 (Validate): 70-80 hrs, \$50-100
- Phase 3 (Launch): 70 hrs, \$200
- Phase 4 (Growth): 100-120 hrs, \$500-600

HONEST TIMELINE EXPECTATION

By Week 4: "Exploit chains are working!"

By Week 8: "Customers like this, willing to pay"

By Week 12: "First customer paid, we're a real business"

By Week 20: "3-5 customers, \$2-5K MRR, validated model"

By Month 6 (Week 24): Decision: Continue full-time, or close?

IF YOU RUN OUT OF TIME

What to cut first (least valuable):

1. GitHub OAuth UI → Use CLI only
2. Advanced governance → Use basic logs
3. Dashboard → Email reports instead
4. On-prem → SaaS only
5. Patch sandbox → Manual approval

What to keep (non-negotiable):

1. Exploit chain discovery
2. Haiku/Sonnet analysis
3. CLI tool
4. Case studies + credibility
5. Customer conversations

SUCCESS CRITERIA (Part-Time)

By Week 4:

- ■ Claude analyzer works
- ■ Found 1+ real chains on public repos
- ■ Cost tracking (prove economics work)

By Week 8:

- ■ 3+ people interested in product
- ■ Feedback analyzed
- ■ Product roadmap adjusted

By Week 12:

- ■ Public launch (HN post)
- ■ 1-2 paying customers
- ■ \$300-999/mo recurring revenue

By Week 20:

- ■ 3-5 paying customers
- ■ \$1-3K MRR
- ■ Clear decision: continue full-time or move on?

THE HARD DECISION AT WEEK 20

At end of Week 20, you'll have:

- Working product ✓
- 3-5 customers ✓
- \$1-3K MRR ✓
- Clear market signal ✓

Then you decide:

OPTION A: Go Full-Time

- Leave day job
- Hire co-founder (Week 20)
- Scale to \$10K+ MRR
- Raise capital
- Compete seriously

OPTION B: Keep Part-Time

- Maintain 2-3 customers
- \$1-3K MRR indefinitely
- Build sustainably
- Don't need external capital
- Stay sane

OPTION C: Sell or Shut Down

- Someone wants to acquire? Sell.
- Market feedback negative? Shut down.
- You realized you don't want this? Move on.
- No shame, you validated the idea.

All three are valid wins.

COMPARISON: Why This Matters

Full-Time Founder Path (12 weeks):

Pros: Speed, intensity, network effects

Cons: Burnout risk, no income, all-in bet

Result: 5 customers OR failure

Part-Time Path (20 weeks):

Pros: Sustainable, low risk, validation, keep day job

Cons: Slower, harder to compete on speed

Result: 3-5 customers + clear signal (yes/no/maybe)

Part-time is better if:

- You have financial stability
- You want to validate first
- You can't afford total risk
- You like sustainable pace

Full-time is better if:

- You have 6+ months savings
- You're confident in the idea
- You want to move fast
- You can handle intensity

You're wisely starting part-time. Smart.

KEY MINDSET

Part-time success $\neq$ full-time success (slower)

Part-time success = full-time success (different path)

- Same product
- Same market validation
- Same revenue potential
- Just takes 20 weeks instead of 12

The question isn't: "Can I do this part-time?"

The answer is: "Will I stick with it for 20 weeks?"

That's the real challenge.

FINAL ADVICE FOR PART-TIME EXECUTION

1. Block time, don't count hours

- Schedule 4-5 hour blocks (e.g., Tuesday 6-10pm)
- Don't switch contexts
- Deep work only

2. Protect momentum

- Week 1: Get API working (confidence)
- Week 2-4: See chains working (momentum)
- Week 5-8: Customer feedback (validation)
- Week 9-12: Revenue (proof)
- Week 13-20: Scaling (compounding)
- Don't let phases blur together

3. Celebrate small wins

- Week 1: API working = BIG WIN
- Week 2: First Haiku call = BIG WIN
- Week 4: Real chains found = BIG WIN
- Week 8: First customer interested = BIG WIN
- Week 12: First payment = HUGE WIN

4. Be ready to pivot

- If chains don't work $\rightarrow$ pivot to faster vulnerability scanning
- If no customers care $\rightarrow$ pivot to different market
- If burnout looming $\rightarrow$ pivot to side project mode
- Pivots are fine, abandoning is fine, persisting is fine

5. Track metrics ruthlessly

- Cost per repo scanned
- Time per chain discovery
- Customer feedback themes
- Revenue runway
- Your energy level
- Make weekly decision: continue or stop?

You're ready. You have the plan. You know the constraints. You know the timeline.

Now it's execution.

Start with Anthropic API setup. Do it today. 30 minutes.

Once that's done, everything else follows.

Document created: April 17, 2026

Your start date: April 18-19, 2026

Your end date: ~August 10, 2026 (20 weeks)

Real decision date: August 10, 2026 (go full-time? or not?)

You're Ready

You have everything you need to succeed:

- ✓ Clear 20-week roadmap with realistic milestones
- ✓ Adaptive access model (GitHub OAuth + CLI + On-Prem)
- ✓ IP protection strategy to keep your moat safe
- ✓ Competitive analysis showing your differentiation
- ✓ Weekly time allocation (20-30 hrs/week is sustainable)
- ✓ Go/no-go decision points at Week 4, 8, 12, and 20

The only thing left is execution.

Start with Anthropic API setup today. If you get that done, everything else follows.

Good luck. You've got this. ■